

Resource and Contention-Aware Concurrent Edge Triage for Astronomical Transient Candidates

Category: R&D

Submitted to: Dr. Mateen Yaqoob
Edge Computing and Intelligent Systems

Umal Khan (23i-0021)
Mohammad Ibrahim (23i-0083)
Muhammad Humza (23i-0090)

Abstract

Time-domain astronomical surveys generate far more transient candidates than can be centrally processed, which makes automated real-bogus triage essential. Existing solutions such as braai, ALeRCE, MeerCRAB, and GOTO's Bayesian CNN rely on static, single-model classifiers, while systems like Edge SpAIce apply one distilled model to a fixed edge deployment with no runtime adaptation. Separately, work on adaptive model selection for embedded systems and on contention-aware, multi-tenant DNN inference scheduling has shown how to allocate compute across competing jobs, but these ideas have not been coupled to astronomical triage or to a recall-constrained data-fate decision. This project proposes a resource- and contention-aware triage pipeline for astronomical transient candidates that couples multi-tier model selection (Tiny, Medium, and Large) with dynamic concurrency admission, tier-dependent discard thresholds, and a real-candidate preservation constraint. Rather than a fixed decision boundary or a fixed number of parallel workers, the system treats a given confidence score as more or less trustworthy depending on which model tier produced it, and it decides at runtime not only which tier should process a candidate but also whether another inference can safely be launched alongside it, given current CPU, GPU, RAM, and queue pressure. Candidates that remain ambiguous after the initial pass are escalated to a stronger tier with MC-dropout uncertainty checks before a discard, compress, or transmit decision is committed. The system will be validated on the public MeerCRAB dataset and evaluated primarily through software-in-the-loop, resource-constrained edge simulation, with deployment on real hardware, preferably a Jetson Orin Nano or, as a CPU-only alternative, a Raspberry Pi 5, pursued as a second phase if such hardware becomes accessible. The central claim is not that the model classifies transients more accurately than existing work, but that the coupled tiering-and-concurrency system preserves a fixed real-transient recall target while reducing compute cost, latency, queueing delay, and bandwidth relative to static and partially adaptive baselines. This will be demonstrated through controlled ablations that isolate the individual contribution of model tiering, dynamic concurrency, and tier-dependent thresholds, rather than crediting the improvement to the presence of multiple tiers alone.

1. Introduction

Time-domain astronomical surveys generate far more transient candidates than can be centrally processed, which makes automated real-bogus triage essential. Existing solutions such as braai, ALeRCE, MeerCRAB, and GOTO's Bayesian CNN rely on static, single-model classifiers, while systems like Edge SpAIce apply one distilled model to a fixed edge deployment with no runtime adaptation. Separately, work on adaptive model selection for embedded systems and on contention-aware, multi-tenant DNN inference scheduling has shown how to allocate compute across competing jobs, but these ideas have not been coupled to astronomical triage or to a recall-constrained data-fate decision. This project proposes a resource- and contention-aware triage pipeline for astronomical transient candidates that couples multi-tier model selection (Tiny, Medium, and Large) with dynamic concurrency admission, tier-dependent discard thresholds, and a real-candidate preservation constraint. Rather than a fixed decision boundary or a fixed number of parallel workers, the system treats a given confidence score as more or less trustworthy depending on which model tier produced it, and it decides at runtime not only which tier should process a candidate but also whether another inference can safely be launched alongside it, given current CPU, GPU, RAM, and queue pressure. Candidates that remain ambiguous after the initial pass are escalated to a stronger tier with MC-dropout uncertainty checks before a discard, compress, or

transmit decision is committed. The system will be validated on the public MeerCRAB dataset and evaluated primarily through software-in-the-loop, resource-constrained edge simulation, with deployment on real hardware, preferably a Jetson Orin Nano or, as a CPU-only alternative, a Raspberry Pi 5, pursued as a second phase if such hardware becomes accessible. The central claim is not that the model classifies transients more accurately than existing work, but that the coupled tiering-and-concurrency system preserves a fixed real-transient recall target while reducing compute cost, latency, queueing delay, and bandwidth relative to static and partially adaptive baselines. This will be demonstrated through controlled ablations that isolate the individual contribution of model tiering, dynamic concurrency, and tier-dependent thresholds, rather than crediting the improvement to the presence of multiple tiers alone.

2. Literature Review

In astronomy, there has been a lot of research on real and bogus classification and efficient deep-learning inference has been studied separately in the field of edge computing. [1] were interested in this problem in astronomy, and created MeerCRAB to classify real astrophysical sources as transient candidates, and to determine how often the MeerLICHT telescope generates false candidates. More recent work by Bonnet-Guerrini et al. furthered the real-bogus classification framework to scalable CNN architectures and uncertainty quantification. At the same time, the concept of edge-computing has gained attention for how to minimize inference latency on resource-poor devices through adaptive model selection and concurrent DNN execution. [3] showed how to dynamically switch between multiple neural networks deployed on an embedded platform, and [2] introduced Quilt for managing multiple concurrent multi-model inferences when memory or contention limits are a concern with GPUs. These are the astronomical, model-selection, uncertainty, and concurrency foundations necessary for the proposed system.

Author(s) & Year	Problem	Methodology	Dataset	Evaluation	Limitations
Hosenie et al. [1], 2021 – MeerCRAB	Automatic separation of real astronomical transient candidates from bogus detections.	Trained multiple CNN architectures on different combinations of astronomical images and used two labeling strategies: thresholding and latent class modeling.	MeerLICHT optical transient candidates containing image-based and numerical features.	Achieved an accuracy of 99.5% on the best model.	The system mainly focuses on classification accuracy; adaptive model selection and edge part is missing.

Bonnet-Guerrini et al., 2026	Performed real and bogus classification without depending on human labeling and also provide uncertainty score as by product.	Introduced scaleable CNN architecture with conv depth increasing as $F \rightarrow 2F \rightarrow 4F$, synthetic transient injection, asymmetric co teaching etc.	Astronomical real and bogus candidates with injected transient examples.	Robustness of classification evaluated under label contamination and comparison of uncertainty-calibration methods; a hybrid UQ method was proposed and the results showed this method to be competitive for calibration.	The full training framework is more complicated and is used to train noisy/unlabelled data instead of edge scheduling. This is a preprint on the arXiv, and is not yet published in a peer-reviewed journal.
Taylor et al. [3], 2018	Selecting an appropriate DNN on resource-limited embedded systems while balancing accuracy and inference time.	Offline-trained prediction model dynamically selects one of several pretrained DNNs according to input and optimization constraints.	ImageNet ILSVRC-2012 validation data evaluated on an NVIDIA Jetson TX2 .	It improved accuracy by 7.52% over the best-performing single-model method and achieved almost $1.8\times$ decrease in inference time.	General image classification and model selection without taking into account concurrent inference, queue pressure, uncertainty or astronomical data.

Lee et al. [2], 2026 – Quilt	Efficient concurrent execution of multiple DNNs on a single GPU without causing memory exhaustion or excessive scheduling overhead.	Peak-memory-aware model partitioning and task-level scheduling using the Quilt compiler and scheduler.	Multi-model DNN inference on GPUs, including Jetson Orin Nano and RTX 4090 platforms.	Compared to model-level and layer-level scheduling, respectively, reduced latency by 25.4% and 37.7% without triggering OOM errors, and compared to NVIDIA Triton, achieved up to 10.8% lower latency.	Requires model partitioning/compiler support and optimized primarily for GPU memory and latency and neglects scientific recall, model confidence, and astronomical data-fate decisions.
------------------------------	---	--	--	--	---

The astronomy studies show that the real–bogus classification system is already effective using CNN, otherwise it would be nothing new to create other classifiers. MeerCRAB gives the task and the context of the observations, and Bonnet-Guerrini et al. give the ability to build a scalable model family and uncertainty estimation, which are useful for building Tiny, Medium and Large classifiers. Both, however, do not explore how to dynamically schedule multiple model capacities to handle varying edge resource situations. On the other hand, Taylor et al. demonstrate that adaptive model selection can save inference cost, but they are not considering concurrent workload or contention when executing a selected model.

Understanding this concurrency problem, Quilt demonstrates that it is possible to benefit from simultaneous DNN inference in terms of efficient resource usage, but also that it can lead to memory pressure and performance degradation if not managed. The suggested project is not about to duplicate any single system, but rather utilizes the complementary discoveries. It features real–bogus classification, model tiers, escalation based on uncertainty, model selection based on the available resources, and contention-based concurrent inference under an "always recall under real transient" constraint. The scientific use case for this is then the "astronomical edge triage" research gap which seeks to maximise classification accuracy, whilst also preserving the scientific recall in the process, while minimising compute, latency, queueing, and communication cost.

3. Problem Statement

Existing real-bogus classifiers rely on static, resource-blind thresholds that treat all model tiers equally, leaving a critical gap between efficient edge deployment and the scientific requirement of reliably preserving rare astronomical transients.

4. Research Objectives & Questions

Primary research question: Can a resource- and contention-aware multi-tier edge inference scheduler dynamically select model capacity and safe concurrency for astronomical transient candidates, while maintaining a fixed real-transient recall target and reducing resource cost, queueing delay, and transmitted data relative to static and non-adaptive baselines?

- **Secondary Research Questions:**

- Model-tier question: can most easy candidates be processed by cheaper CNN tiers, while ambiguous candidates are escalated to stronger models?
- Concurrency question: can dynamic concurrency improve throughput and queue stability compared with purely sequential inference, without causing unacceptable contention?
- Recall question: can the system maintain a target real-transient survival recall while reducing compute and bandwidth?
- Coupling question: do tier-dependent decision thresholds provide a measurable benefit over a single fixed threshold applied across all tiers?
- Uncertainty question: does MC-dropout uncertainty identify candidates that are disproportionately likely to be misclassified, and therefore worth escalating?
- Burst-load question: does queue-aware adaptation improve end-to-end latency and throughput specifically during candidate-arrival bursts?

Hypotheses:

- **H1, adaptive tiering:** an adaptive Tiny, Medium, and Large strategy reduces average inference cost relative to always using the Large model, while maintaining approximately the same target recall.
- **H2, dynamic concurrency:** resource-aware dynamic concurrency improves throughput and reduces queue growth compared with sequential processing, especially during bursty arrivals.
- **H3, contention awareness:** unrestricted parallel inference eventually degrades latency or stability, whereas concurrency admission based on CPU, GPU, RAM, and queue state maintains a better operating region.
- **H4, tier-aware trust:** tier-dependent thresholds preserve more real candidates at equal resource cost than identical thresholds applied across Tiny, Medium, and Large models.
- **H5, uncertainty-guided escalation:** ambiguous or high-uncertainty candidates contain a higher fraction of model errors than low-uncertainty candidates, making selective escalation more efficient than running the Large model on every candidate.

Measurable objectives:

1. **Exploratory data analysis:** quantify class imbalance, cutout channel statistics, and label-threshold effects in MeerCRAB (T9 versus T10 labeling), and visualize real versus bogus examples across the New, Reference, Difference, and Scorr channels.
2. **Model development:** implement the F, 2F, 4F CNN architecture at three filter widths (Tiny, Medium, Large), train each tier to convergence on MeerCRAB, and implement the MC-dropout uncertainty head.

3. **Handling class imbalance:** apply and compare imbalance-handling strategies, such as class-weighted loss, focal loss, or oversampling, and report their effect on PR-AUC and recall at a fixed false-positive rate, since raw accuracy is not meaningful under severe imbalance.
4. **Adaptive scheduling implementation:** build the Context Monitor, the Adaptive Controller for tier selection, the ambiguous-zone escalation logic, and the tier-dependent threshold mapping described in the methodology below.
5. **Concurrency and contention-handling:** design and implement the dynamic concurrency admission controller that decides, in addition to which tier to use, whether another inference may safely be launched given current resource headroom, and characterize the point at which added concurrency stops helping and starts hurting.
6. **Queue and burst-load testing:** build a workload generator that produces steady and bursty candidate-arrival patterns, and evaluate queue growth, latency, and throughput under each pattern.
7. **Validation of results:** run the six-system comparison described in the methodology, each calibrated to a fixed recall target on the validation set, then measured for cost on the held-out test set, producing the recall-versus-cost Pareto plot as the headline result.
8. **Dashboards and reporting artifacts:** build a logging and visualization dashboard showing tier-usage distribution, escalation rate, concurrency level, queue behavior, and recall or cost trade-offs, either in real time or from logged runs, to make system behavior interpretable during development and for the final presentation.

5. Dataset Description

The planned primary dataset is MeerCRAB, associated with the MeerLICHT survey [2], publicly available through Zenodo and requiring no institutional access. It consists of real telescope candidate data with binary ground-truth labels, where 0 denotes bogus (an artifact) and 1 denotes real (a genuine astrophysical transient). Each candidate is represented as an image cutout with up to four channels, New, Reference, Difference, and Scorr, the last being a significance or signal-to-noise representation of the difference image. The dataset has been released under more than one labeling threshold, referred to here using the T9 and T10 notation rather than the earlier `threshold_9` and `threshold_10` phrasing, reflecting different strictness levels in how a candidate was originally annotated as real. A reference codebase from the original MeerCRAB authors is available on GitHub, including training and prediction scripts, a demonstration notebook, and pretrained model weights, which will inform baseline architecture and preprocessing decisions.

This dataset description should be treated as a starting point to be verified and frozen, not as a settled specification. Before implementation begins, the following must be confirmed directly against the current downloadable release rather than assumed:

- the exact downloadable dataset record and version, and its licensing terms;
- the exact train and test files provided, the number of candidates, and the class counts and resulting imbalance;
- which labeling threshold, T9 or T10, is used, and the exact image dimensions and channel order;
- whether the data contain missing values, malformed samples, or undocumented metadata fields;
- the normalization method used in the original study, for comparability with prior baselines;

- whether the provided splits are truly held out, or whether a validation split still needs to be carved from the training data;
- whether multiple detections can originate from the same underlying astronomical object, in which case splitting may need to be performed at the object level rather than the candidate level, to avoid leakage between train, validation, and test sets;
- the storage size per candidate, and the raw versus compressed byte cost, since these figures are required directly for the bandwidth-cost experiments described in the methodology.

Pending that verification, the working split policy is to preserve the official held-out test split if one is supplied, to create a stratified validation split only from the training data, and never to tune decision thresholds or ambiguous-zone bounds using the test set.

6. Proposed Methodology

6.2 Architecture

The system performs binary real-bogus classification, producing a score used for a discard, compress, or transmit decision, trained and evaluated on the public MeerCRAB dataset using the provided train and test splits and a validation set carved from the training data. Before any modeling begins, the dataset verification checklist in Section 5 will be executed as an explicit first methodological step, since decisions such as the labeling threshold used, the exact channel order, and whether object-level splitting is required all affect how the rest of the pipeline is built and evaluated.

6.2 Architecture

A single CNN design, three convolutional blocks with filter counts F , $2F$, and $4F$, followed by fully connected layers and a sigmoid output, is scaled by filter width to produce three tiers: Tiny (F equal to 8), Medium (F equal to 16), and Large (F equal to 32, with an added MC-dropout uncertainty head). These widths are illustrative starting points, informed by braai, ALerCE, MeerCRAB, and the Bonnet-Guerrini architecture, and will be finalized only after profiling parameter count, validation performance, latency, and memory footprint for each candidate width.

6.3 Pipeline

A Context Monitor, built on psutil and related tooling, continuously reports CPU, RAM, GPU, queue, and bandwidth conditions to an Adaptive Scheduler. The scheduler makes two coupled decisions rather than one: it selects a model tier for an incoming candidate, and it decides whether another inference may be admitted to run concurrently given current resource headroom, active jobs per tier, and queue pressure. Candidates with confidently high or low scores are resolved immediately at their assigned tier; candidates that fall into an ambiguous zone are escalated to a stronger tier and re-scored using MC-dropout, which yields both a second-pass score and an uncertainty estimate. The final score is then mapped to a discard, compress, or transmit decision using thresholds that shift depending on which tier produced the score, on the reasoning that a cheaper tier's confidence is less trustworthy at the same nominal value than a stronger tier's.

6.4 Experimental baselines

Six systems will be compared, each independently calibrated on the validation set to reach a fixed real-transient recall target, for example 95 percent, before being evaluated once on the held-out test set:

- B1, static Large, sequential: always the Large model, with concurrency fixed at one; the accuracy-quality upper baseline.
- B2, static Tiny, sequential: always the Tiny model, with concurrency fixed at one; the cheap lower baseline.
- B3, adaptive tier, sequential: dynamic Tiny, Medium, or Large selection, but no parallel inference; isolates the model-selection gain on its own.
- B4, adaptive tier with fixed concurrency: dynamic model selection combined with a constant worker or concurrency setting; isolates whether concurrency itself helps, independent of whether it is resource-aware.
- B5, adaptive tier with dynamic concurrency: resource-aware model-tier selection combined with resource-aware concurrency admission; tests the contribution of the scheduler as a whole.
- B6, the proposed full system: adaptive tier selection, dynamic concurrency, uncertainty-based escalation, tier-dependent thresholds, and a recall-constrained data-fate policy, together.

This six-way comparison is a stricter isolation of which mechanism actually drives any observed gain than a simple two- or four-way comparison would allow, since it separates the effect of tiering (B1 to B3), the effect of concurrency (B3 to B5), and the effect of tier-dependent thresholds and uncertainty together (B5 to B6).

6.5 Ablation studies

- Ablation A, tier selection: adaptive tiering switched on versus off.
- Ablation B, concurrency: sequential versus fixed concurrency versus adaptive concurrency.
- Ablation C, tier-aware thresholds: a single common threshold versus per-tier thresholds.
- Ablation D, uncertainty: no uncertainty quantification versus uncertainty-driven escalation.
- Ablation E, queue awareness: a resource-only scheduler versus a resource-and-queue-aware scheduler.
- Ablation F, optional: a simple memory-headroom admission rule versus a Quilt-inspired memory-aware admission policy, or a simple single-objective scheduler versus a Convergo-inspired multi-constraint priority scheme; implemented only if time permits.

6.6 Hardware and simulation strategy

No physical edge hardware is currently available to the project team. The primary evaluation environment will therefore be a controlled, software-in-the-loop, resource-constrained edge simulation, built from Docker or Linux cgroup limits, stress-ng for CPU, memory, and I/O contention, and tc combined with netem for bandwidth, latency, and jitter shaping, together with SimPy for discrete-event simulation of candidate arrivals and queueing. It is important to be precise about what this environment can and cannot claim: a constrained x86 machine can approximate limited CPU, RAM, worker counts, queueing behavior, and network conditions, but it cannot faithfully reproduce ARM execution, Jetson GPU

microarchitecture, CUDA scheduling details, or real thermal and power behavior. This project will therefore describe its primary environment as software-in-the-loop resource-constrained edge simulation, and will not describe it as hardware emulation of a Jetson or Raspberry Pi device. If physical hardware becomes accessible during the project, a second validation phase will attempt deployment on a Jetson Orin Nano, which is preferred because it matches the device used in the Quilt evaluation, with a Raspberry Pi 5 as a CPU-only alternative if a Jetson-class device is not available. Hardware deployment would strengthen the final results but is treated as a second phase rather than a precondition for a meaningful evaluation.

7. Tools and Technologies

Pre-hardware, software-in-the-loop validation (to test controller logic before physical deployment):

- **Docker** with `--cpus/--memory` limits (or raw Linux cgroups) to simulate a resource-constrained edge device.
- **stress-ng** to inject configurable, repeatable CPU/memory/IO contention and test controller behavior under load.
- **tc + netem** to inject controlled bandwidth, latency, and jitter conditions on a network interface.
- **SimPy** for discrete-event simulation of candidate arrivals (e.g., Poisson bursts) and queueing, enabling large-scale ablations without repeatedly running real inference.

Core implementation:

- **PyTorch** for the F-parameterized CNN architecture and MC-dropout uncertainty estimation.
- **psutil** for cross-platform CPU/RAM monitoring by the Context Monitor.
- **asyncio / ThreadPoolExecutor / ProcessPoolExecutor** for concurrent in-flight inference across model tiers.
- **pandas** for decision logging (timestamp, tier used, resource snapshot, uncertainty, action taken, ground truth).
- **matplotlib / seaborn** for Pareto curves and ablation plots; a lightweight dashboard (e.g., Streamlit) for real-time or post-hoc visualization of tier usage, escalation rate, and recall/cost trade-offs.
- **Git + YAML configs** for experiment tracking, sufficient given the project's scale.

Hardware:

- **Raspberry Pi 4/5 (4–8GB)** as the primary edge device — a practical choice for a 2–3 month timeline, avoiding Jetson/CUDA/JetPack setup risk.
- **NVIDIA Jetson Nano/Orin Nano** as a stretch-goal edge device, if accessible via a university lab, for a more realistic GPU-based edge deployment.
- A laptop/PC or free-tier cloud VM as the "cloud" endpoint to receive transmitted candidates.
- A real WiFi/Ethernet network link shaped with **tc/netem** for reproducible bandwidth/latency conditions.

- (Optional) A USB power meter or **tegrastats** for energy-per-inference measurement, as a nice-to-have rather than a core metric.

8. Evaluation Metrics

The following metrics constitute a provisional evaluation scheme to assess model accuracy, uncertainty calibration, and system-level trade-offs. The precise subset and primary optimization targets will be formally decided during system benchmarking.

Per-model classification metrics (evaluated per tier: Tiny, Medium, Large):

- Precision, Recall, and F1, with recall weighted as the priority metric given the cost asymmetry of missing a real transient.
- False Positive Rate at a fixed missed-detection rate (e.g., FPR @ 1% missed detections) — the standard reporting convention in this literature (braai, ALerCE).
- ROC-AUC and PR-AUC, with PR-AUC weighted more heavily given the severe class imbalance.
- Calibration quality via reliability curves and Brier score, since threshold-based decisions depend on scores being meaningfully calibrated, not just rank-ordered.

Uncertainty-quality metrics:

- Correlation between MC-dropout uncertainty and actual classification error, do high-uncertainty candidates actually correspond to mistakes?
- Escalation rate, the percentage of candidates falling into the ambiguous zone and requiring a second-tier pass.

System-level metrics:

- **Scientific recall** percentage of true real/important transients that survive to transmission; the hard constraint held fixed across all compared systems.
- **Data reduction rate** percentage of candidates discarded.
- **Compute cost saved** inference time/FLOPs per candidate relative to an always-Large baseline, including context-monitoring, controller-decision, and MC-dropout escalation overhead (not model inference alone).
- **Bandwidth/energy saved** bytes transmitted and (optionally) energy per candidate, relative to the static baseline.
- **Tier usage distribution** percentage of candidates resolved at each tier, as evidence the system is genuinely adaptive rather than defaulting to one tier.
- **End-to-end decision latency** measured under simulated bursty load/contention, not just isolated single-candidate latency.

Headline result: A recall-vs-cost Pareto curve plotting scientific recall (y-axis) against resource cost, compute or bandwidth (x-axis), for the four compared systems (static Large-only, static Tiny-only, adaptive-uncoupled, adaptive-coupled). The adaptive-coupled curve sitting closer to the top-left (high

recall, low cost) than the uncoupled curve is the direct evidence that tier-dependent thresholds, not just the presence of multiple tiers, drive the improvement.

9. References

- [1] Z. Hosenie, "MeerCRAB: MeerLICHT classification of real and bogus transients using deep learning," *Exp. Astron.*, vol. 51, no. 2, pp. 319–344, 2021, doi: 10.1007/s10686-021-09757-1.
- [2] J. Lee, "Peak-memory-aware partitioning and scheduling for multi-tenant DNN model inference," *J. Syst. Archit.*, vol. 173, p. 103696, 2026, doi: 10.1016/j.sysarc.2026.103696.
- [3] B. Taylor et al., "Adaptive deep learning model selection on embedded systems," *ACM SIGPLAN Not.*, vol. 53, no. 6, pp. 31–43, 2018, doi: 10.1145/3211332.3211336.